

NestJS CRUD - Inicio proyecto con Node

Perfecto, vamos a empezar **desde 0 y por fases**, siguiendo el documento maestro. La regla será: **no pasamos a la siguiente fase hasta que esta fase quede validada**. En el documento, la **Fase 1** es preparar herramientas y validar entorno: Node.js, npm, MySQL, VS Code y Nest CLI.

Fase 1: Validar entorno

Como ya tienes:

Node.js 24

MySQL con Laragon

VS Code

Ahora vamos a validar que todo responda bien.

Abre **VS Code** y luego abre una terminal:

Terminal > New Terminal

Ejecuta estos comandos uno por uno:

</> Bash

```
node -v
```

Debe mostrar algo como:

</> Bash

Luego:

</> Bash

```
npm -v
```

Debe mostrar una versión de npm.

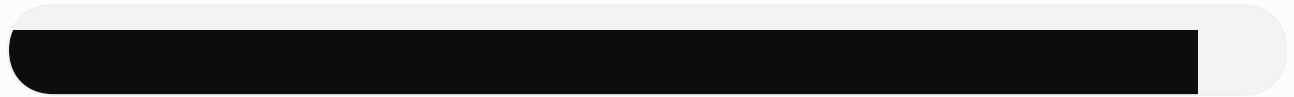
Ahora validamos MySQL:

</> Bash

```
--version
```

Si te responde una versión de MySQL, perfecto.

Si te sale que `mysql` no se reconoce, no te preocupes. En Laragon muchas veces MySQL funciona, pero no está agregado al PATH de Windows. En ese caso puedes abrir la terminal desde:

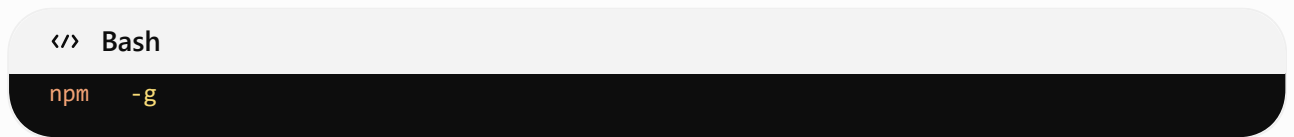


Y ahí probar de nuevo:

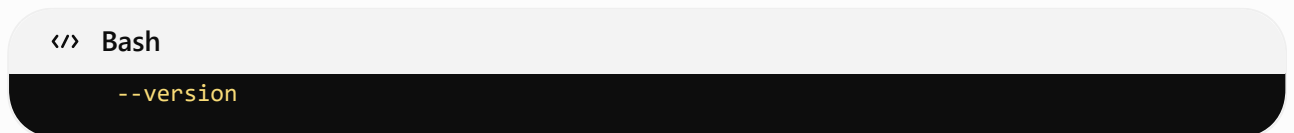


Instalar Nest CLI

Ahora instala NestJS CLI de forma global:



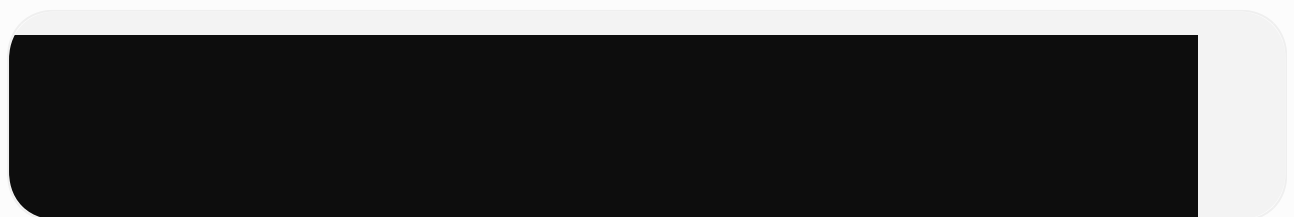
Cuando termine, valida:



Debe mostrar una versión de Nest CLI.

Checklist de esta fase

Antes de avanzar, necesitamos confirmar esto:



Cuando tengas esos resultados, mándame una captura o copia lo que te sale en consola y seguimos con la **Fase 2: Crear el proyecto NestJS desde cero**.

Genial, continuamos con la **Fase 5: configurar Swagger y validación global**.

En esta fase activaremos:

Swagger en:

Y el prefijo global:

El documento maestro indica que el entregable verificable es que Swagger quede disponible en `/api/docs` y que el prefijo global `/api` quede activo.

1. Modificar `src/main.ts`

Abre este archivo:

Reemplaza todo su contenido por esto:

`</>` TypeScript

```
import ValidationPipe from '@nestjs/common'
import ConfigService from '@nestjs/config'
import NestFactory from '@nestjs/core'
import { DocumentBuilder, SwaggerModule } from '@nestjs/swagger'
import AppModule from './app.module'

async function bootstrap() {
  const app = await NestFactory.create(AppModule)

  const configService = app.get(ConfigService)

  app.setGlobalPrefix('api')

  app.use(
    new ValidationPipe({
      whitelist: true,
      transform: true,
    })
  )

  const swaggerConfig = new DocumentBuilder()
    .setTitle('CRUD Productos API')
    .setDescription('API REST con NestJS, MySQL, Drizzle ORM, Swagger y JWT')
    .setVersion('1.0')
    .build()

  const document = SwaggerModule.createDocument(app, swaggerConfig)

  SwaggerModule.setup('api/docs', app, document)
```

```
const port = configService.    number 'PORT' ?? 3000

await app.    port

console.    `[APP] Servidor ejecutándose en http://localhost:  port /api`
console.    `[APP] Swagger disponible en http://localhost:  port /api/docs`

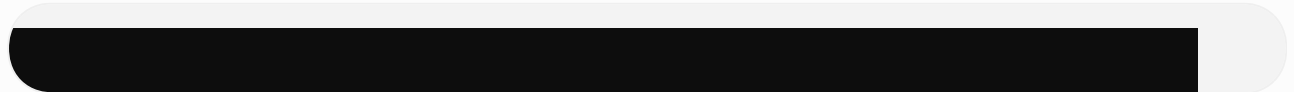
bootstrap
```

2. Levantar el servidor

Ejecuta:

```
</> Bash
npm    start
```

Ahora abre:



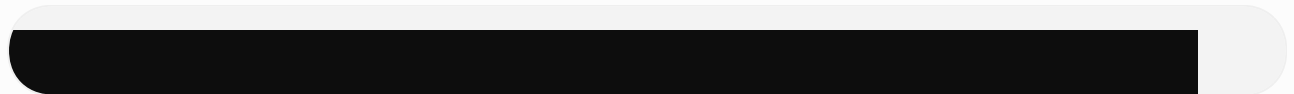
Debe abrir Swagger.

3. Importante

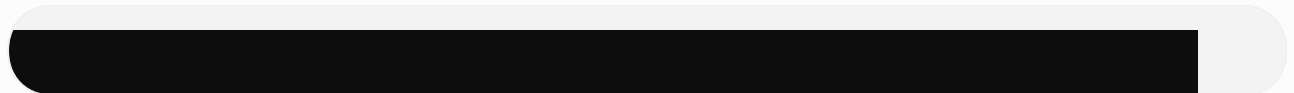
Desde esta fase, como activamos:

```
</> TypeScript
app.    'api'
```

La ruta inicial ya no será:



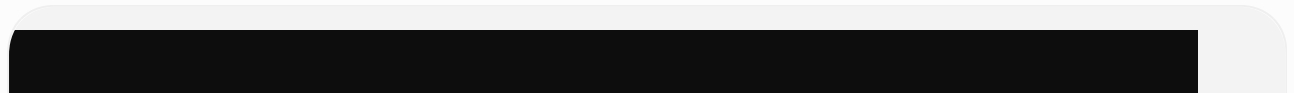
Ahora será:



Si entras a `http://localhost:3000` y ves error `404`, no pasa nada. Es normal por el prefijo global.

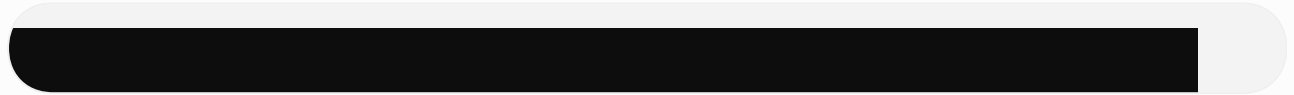
Validación de esta fase

Confirma que se cumple esto:

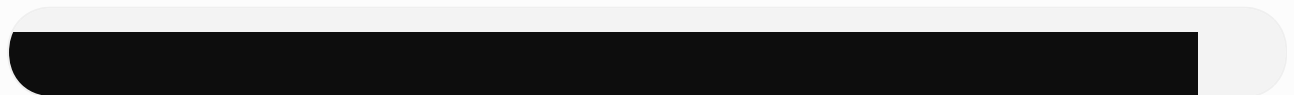


Cuando Swagger abra correctamente, seguimos con la **Fase 6: configurar Drizzle ORM y MySQL**.

Sí, ahí está el problema. El código que pegaste **no debe estar en** `src/app.module.ts` . Ese código pertenece a:



Por eso te sale:



Porque tu `app.module.ts` está exportando esto:

❏ TypeScript

```
export class AuthModule
```

cuando debería exportar:

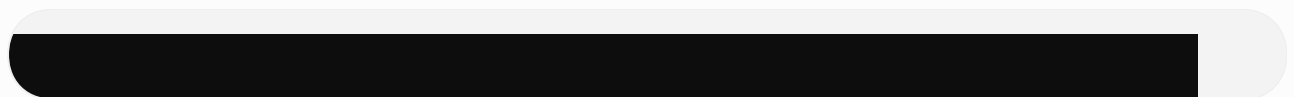
❏ TypeScript

```
export class AppModule
```

El documento maestro separa `AppModule` como módulo principal y `AuthModule` como módulo de autenticación.

1. Corrige `src/app.module.ts`

Abre:



Déjalo así:

❏ TypeScript

```
import Module from '@nestjs/common'
import ConfigModule from '@nestjs/config'

import AppController from './app.controller'
import AppService from './app.service'
import AuthModule from './auth/auth.module'
import DatabaseModule from './database/database.module'

@Module
```

```

    ConfigModule.
        true
        '.env'

    DatabaseModule
    AuthModule

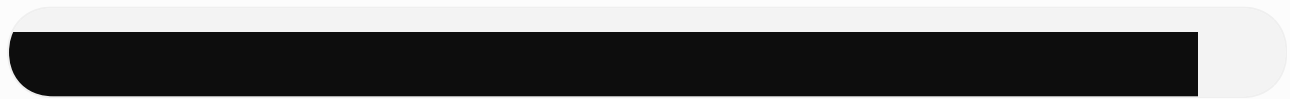
    AppController
    AppService

export class AppModule

```

2. Corrige src/auth/auth.module.ts

Ahora abre:



Ahí sí debe ir el código de JWT, Passport y Auth. Déjalo así:

<> TypeScript

```

import Module from '@nestjs/common'
import ConfigModule, ConfigService from '@nestjs/config'
import JwtModule from '@nestjs/jwt'
import type JwtModuleOptions from '@nestjs/jwt'
import PassportModule from '@nestjs/passport'

import DatabaseModule from '../database/database.module'
import AuthController from './auth.controller'
import AuthService from './auth.service'
import JwtStrategy from './strategies/jwt.strategy'

type JwtExpiresIn = NonNullable JwtModuleOptions 'signOptions' 'expiresIn'

@Module

    DatabaseModule
    PassportModule
    JwtModule.
        ConfigModule
        ConfigService
        configService ConfigService JwtModuleOptions
    const secret = configService. string 'JWT_SECRET'

    if !secret
        throw new Error 'JWT_SECRET no está configurado en el archivo .env'

    const expiresIn = configService. string 'JWT_EXPIRES_IN' ??
        '1d' as JwtExpiresIn

    return

```

```
        AuthController
        AuthService JwtStrategy

export class AuthModule
```

La diferencia importante es esta:

En `app.module.ts` las rutas van así:

</> TypeScript

```
import AuthModule from './auth/auth.module'
import DatabaseModule from './database/database.module'
```

Pero en `auth.module.ts`, como está dentro de la carpeta `auth`, las rutas van así:

</> TypeScript

```
import DatabaseModule from '../database/database.module'
import AuthController from './auth.controller'
import AuthService from './auth.service'
```

3. Reinicia el servidor

Detén el servidor:

</> Bash

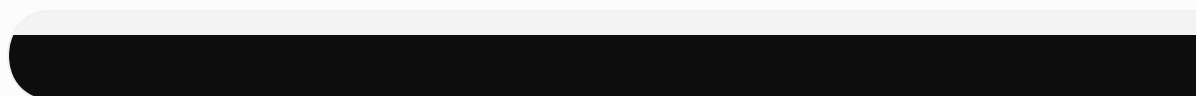
```
+
```

Luego ejecuta:

</> Bash

```
npm start
```

Con eso debería desaparecer el error de:



Tu `main.ts` está bien si tiene esto:

</> TypeScript

```
import AppModule from './app.module'
```

y esto:

```
</> TypeScript
```

```
const app = await NestFactory.      AppModule
```

genial chatpt continuemos

Genial, continuamos con la **Fase 13: Crear módulo Products y DTOs**.

En esta fase vamos a preparar la estructura para el CRUD de productos. Todavía no implementaremos la lógica completa; eso será en la **Fase 14**. Según el documento maestro, aquí debemos crear `ProductsModule`, `ProductsController`, `ProductsService` y los DTOs de creación y actualización.

1. Generar módulo Products

Ejecuta en la terminal:

```
</> Bash
```

Luego:

```
</> Bash
```

```
--no-spec
```

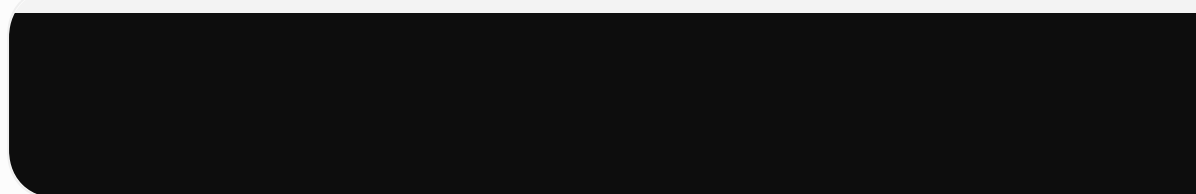
Y luego:

```
</> Bash
```

```
service
```

```
--no-spec
```

Esto debe crear:



2. Crear carpeta `dto`

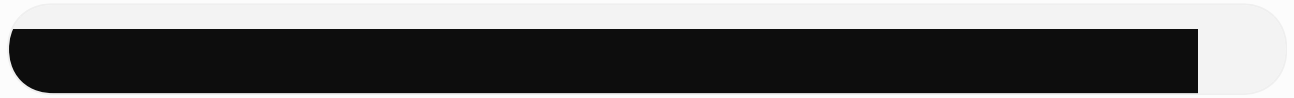
Ejecuta:

```
</> Bash
```

```
mkdir
```

3. Crear `create-product.dto.ts`

Crea este archivo:



Agrega:

«> TypeScript

```
import ApiProperty from '@nestjs/swagger'
import Type from 'class-transformer'
import {
  IsInt,
  IsNumber,
  IsOptional,
  IsString,
  Min,
  MinLength,
} from 'class-validator'

export class CreateProductDto {
  @ApiProperty({ description: 'Teclado mecánico' })
  @IsString
  @MinLength(2)
  keyboard!: string

  @ApiProperty({ description: 'Precio' })
  @Type(() => Number)
  @IsNumber
  @Min(0)
  price!: number

  @ApiProperty({ description: 'Teclado RGB para programación' })
  @IsBoolean
  rgb!: boolean

  @IsOptional
  @IsString
  description?: string

  @ApiProperty({ description: 'Cantidad' })
  @Type(() => Number)
  @IsInt
  @Min(0)
  quantity!: number
}
```

4. Crear `update-product.dto.ts`

Crea este archivo:



Agrega:

⟨⟩ TypeScript

```
import PartialType from '@nestjs/swagger'

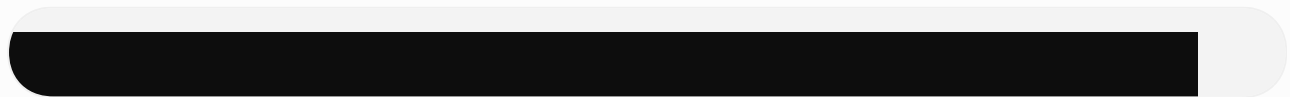
import CreateProductDto from '../create-product.dto'

export class UpdateProductDto extends PartialType(CreateProductDto)
```

Con `PartialType`, todos los campos de `CreateProductDto` pasan a ser opcionales para el `PATCH`.

5. Revisar `products.module.ts`

Abre:



Por ahora puede estar así:

⟨⟩ TypeScript

```
import Module from '@nestjs/common'

import ProductsController from '../products.controller'
import ProductsService from '../products.service'

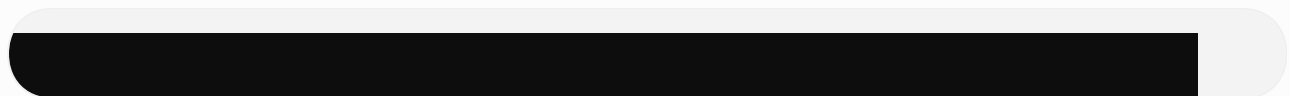
@Module({
  controllers: [ProductsController],
  providers: [ProductsService]
})

export class ProductsModule
```

En la siguiente fase agregaremos `DatabaseModule` porque ahí recién usaremos Drizzle dentro de `ProductsService`.

6. Revisar `app.module.ts`

Abre:



Debe tener `ProductsModule` importado:

⟨⟩ TypeScript

```

import Module from '@nestjs/common'
import ConfigModule from '@nestjs/config'

import AppController from './app.controller'
import AppService from './app.service'
import AuthModule from './auth/auth.module'
import DatabaseModule from './database/database.module'
import ProductsModule from './products/products.module'

@Module

  ConfigModule.
    true
    '.env'

  DatabaseModule
  AuthModule
  ProductsModule

  AppController
  AppService

export class AppModule

```

Si `ProductsModule` no aparece, agrégalo manualmente.

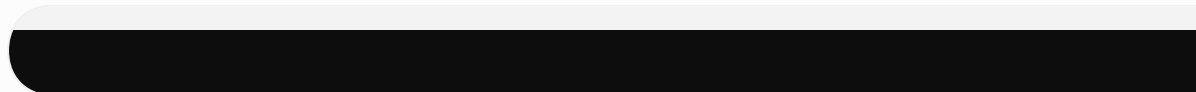
7. Validar que compile

Ejecuta:

```
</> Bash
```

```
npm start
```

Luego abre Swagger:



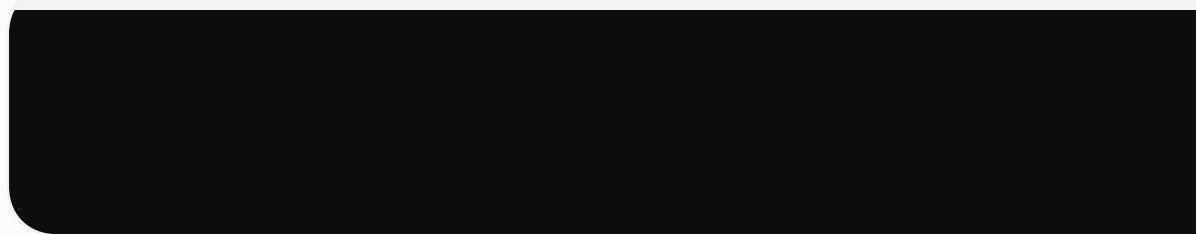
Por ahora todavía no tendremos el CRUD completo visible, porque en la siguiente fase reemplazaremos el controlador y el servicio de productos.

Cuando compile sin errores, pasamos a la **Fase 14: implementar CRUD completo de productos protegido con JWT**.

genial continuemos

Genial, vamos con la **Fase 14: implementar CRUD completo de productos**.

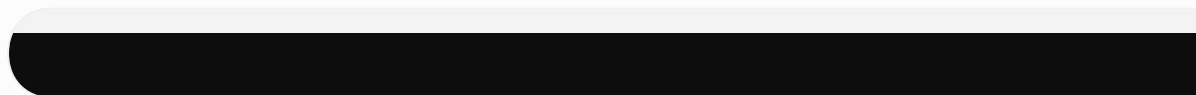
En esta fase crearemos los endpoints protegidos con JWT para:



Según el documento maestro, esta fase debe dejar el CRUD completo visible en Swagger y protegido con `JwtAuthGuard`.

1. Modificar `src/products/products.service.ts`

Abre:



Reemplaza todo por esto:

❏ TypeScript

```
import
  Inject
  Injectable
  InternalServerErrorException
  NotFoundException
  from '@nestjs/common'
import { desc, eq } from 'drizzle-orm'

import DB from '../database/database.provider'
import type Database from '../database/database.provider'
import products from '../db/schema'
import CreateProductDto from '../dto/create-product.dto'
import UpdateProductDto from '../dto/update-product.dto'

@Injectable()
export class ProductsService {
  @Inject DB private readonly db Database

  async create(dto: CreateProductDto) {
    console.log('[PRODUCTS] Creando producto:', dto)

    const inserted = await this.db
      .insert(products)
      .values(dto)
      .onConflictDoUpdate({
        target: products.id,
        set: {
          name: dto.name ?? null,
          price: dto.price ?? null,
        },
      })
      .returning()
  }
}
```

```

    .

    const id = inserted 0

    if !id
        console.log('[PRODUCTS] Error al obtener ID del producto creado')
        throw new InternalServerErrorException 'Error al crear producto'

    console.log('[PRODUCTS] Producto creado con ID:' id

    return this.findById(id)

    async
        console.log('[PRODUCTS] Listando productos')

        return this.products.sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime()).desc products.

    async findById(id: number)
        console.log('[PRODUCTS] Buscando producto ID:' id

        const product = await this.products.findOne({
            _id: id
        })

        if !product
            console.log('[PRODUCTS] Producto no encontrado ID:' id
            throw new NotFoundException 'Producto no encontrado'

        return product

    async update(id: number, dto: UpdateProductDto)
        await this.findById(id)

        console.log('[PRODUCTS] Actualizando producto ID:' id

        const data = {
            name: dto.name !== undefined ? dto.name : undefined,
            price: dto.price !== undefined ? dto.price : undefined,
            description: dto.description !== undefined ? dto.description : undefined,
            stock: dto.stock !== undefined ? dto.stock : undefined,
        }

        if Object.keys(data).length === 0
            console.log('[PRODUCTS] No se enviaron datos para actualizar ID:' id
            return this.findById(id)

```

```

    await this. .      products .      data .      eq products.      id

    return this.      id

async      id number
    await this.      id

    console.      '[PRODUCTS] Eliminando producto ID:' id

    await this. .      products .      eq products.      id

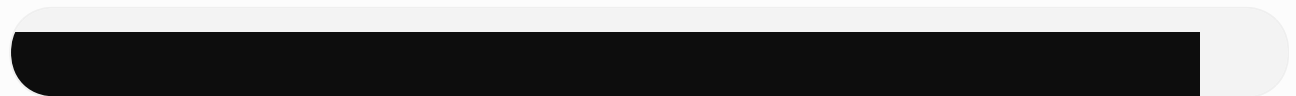
    return

        'Producto eliminado correctamente'

```

2. Modificar `src/products/products.controller.ts`

Abre:



Reemplaza todo por esto:

⌞ TypeScript

```

import
  Body
  Controller
  Delete
  Get
  Param
  ParseIntPipe
  Patch
  Post
  UseGuards
  from '@nestjs/common'
import ApiBearerAuth ApiOperation ApiTags from '@nestjs/swagger'

import JwtAuthGuard from '../auth/guards/jwt-auth.guard'
import CreateProductDto from '../dto/create-product.dto'
import UpdateProductDto from '../dto/update-product.dto'
import ProductsService from '../products.service'

@ApiTags 'Products'
@ApiBearerAuth
@UseGuards JwtAuthGuard
@Controller 'products'

```

```

export class ProductsController {
    private readonly productService: ProductService

    @Post
    @ApiOperation('Crear producto')
    @Body() dto: CreateProductDto
    return this.service.create(dto)

    @Get
    @ApiOperation('Listar productos')
    return this.service.findAll()

    @Get('/:id')
    @ApiOperation('Obtener producto por ID')
    @Param('id') ParseIntPipe id: number
    return this.service.findOne(id)

    @Patch('/:id')
    @ApiOperation('Actualizar producto')

    @Param('id') ParseIntPipe id: number
    @Body() dto: UpdateProductDto

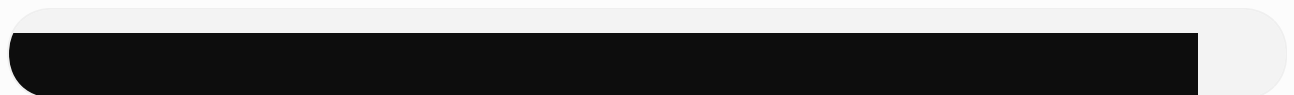
    return this.service.update(id, dto)

    @Delete('/:id')
    @ApiOperation('Eliminar producto')
    @Param('id') ParseIntPipe id: number
    return this.service.remove(id)
}

```

3. Modificar `src/products/products.module.ts`

Abre:



Déjalo así:

`</>` TypeScript

```

import Module from '@nestjs/common'

import DatabaseModule from '../database/database.module'
import ProductsController from './products.controller'

```

```
import ProductsService from './products.service'

@Module
  DatabaseModule
  ProductsController
  ProductsService

export class ProductsModule
```

4. Validar `src/app.module.ts`

Debe tener `ProductsModule` importado:

<> TypeScript

```
import Module from '@nestjs/common'
import ConfigModule from '@nestjs/config'

import AppController from './app.controller'
import AppService from './app.service'
import AuthModule from './auth/auth.module'
import DatabaseModule from './database/database.module'
import ProductsModule from './products/products.module'

@Module
  ConfigModule.
    true
    '.env'

  DatabaseModule
  AuthModule
  ProductsModule

  AppController
  AppService

export class AppModule
```

5. Levantar proyecto

Ejecuta:

<> Bash

```
npm start
```

Abre Swagger:

Ahora debes ver el grupo:

Con estos endpoints:

6. Probar seguridad JWT

Primero prueba:

Sin token debe responder:

JSON

```
"Unauthorized"  
401
```

Luego haz login en:

Con el usuario demo:

JSON

```
"admin@andercode.com"  
"Admin123456"
```

Copia el `access_token` .

En Swagger, clic en:

Pega:

Ahora prueba:

Debe listar los productos del seeder.

7. Body para crear producto

En `POST /api/products` , prueba con:

JSON

```
"Mouse inalámbrico"
59.9
  "Mouse ergonómico para oficina"
20
```

Cuando compile y el CRUD funcione con token, pasamos a la **Fase 15: pruebas completas** desde Swagger por endpoint.